НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО”
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ АТОМНОЇ ТА ТЕПЛОВОЇ ЕНЕРГЕТИКИ

кафедра ЦИФРОВИХ ТЕХНОЛОГІЙ В ЕНЕРГЕТИЦІ

“До захисту допущено”
Завідувач кафедри ЦТЕ
Наталія АУШЕВА

“___”_____2025р.

Дипломна робота

на здобуття ступеня бакалавра
за освітньо-професійною програмою
“Цифрові технології в енергетиці”
зі спеціальності *122 “Комп’ютерні науки”*

на тему: Додаток для децентралізованого голосування на основі блокчейн технологій

Виконав (-ла):
студент (-ка) IV курсу, групи ТР-13

КУЧИНСЬКИЙ Кирил Олегович _____
(прізвище, ім’я, по батькові) (підпис)

Керівник: *асистент каф. ЦТЕ*
ас. Пасічнюк Антон Олексійович _____
(посада, науковий ступінь, вчене звання, прізвище, ім’я, по батькові) (підпис)

Н.контроль: *асистент каф. ЦТЕ*
ас. Волков Олександр Володимирович _____
(посада, науковий ступінь, вчене звання, прізвище, ім’я, по батькові) (підпис)

Рецензент: *професор каф. ІПЗЕ*
д.т.н., проф. Барабаш Олег Володимирович _____
(посада, науковий ступінь, вчене звання, науковий ступінь, прізвище, ім’я, по батькові) (підпис)

Засвідчую, що у цій дипломній роботі немає запозичень з праць інших авторів без відповідних посилань.
Студент (-ка) _____
(підпис)

Київ – 2025

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО”

НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ АТОМНОЇ ТА ТЕПЛОВОЇ ЕНЕРГЕТИКИ

Кафедра ЦИФРОВИХ ТЕХНОЛОГІЙ В ЕНЕРГЕТИЦІ

Рівень вищої освіти – перший (бакалаврський)
спеціальність 122 “Комп’ютерні науки”
Освітньо-професійна програма “Цифрові технології в енергетиці”

ЗАТВЕРДЖУЮ
Завідувач кафедри ЦТЕ
_____ Наталія АУШЕВА_

«__»_____2025р.

ЗАВДАННЯ
на дипломну роботу студенту

КУЧИНСЬКОМУ Кирилу Олеговичу
(прізвище, ім’я, по батькові)

1. Тема роботи Додаток для децентралізованого голосування на основі блокчейн технологій

керівник роботи
Пасічнюк Антон Олексійович, асистент
(прізвище, ім’я, по батькові, науковий ступінь, вчене звання)

затверджений наказом по університету від «___»_____ 2025 р. №_____

2. Термін подання роботи студентом *2025 року*

3. Вихідні дані до роботи персональний комп’ютер з операційною системою Microsoft Windows, середовище розробки IntelliJ IDEA, мова програмування JavaScript, мова програмування Solidity, бібліотека React.js, фреймворк Nest.js.

4. Зміст роботи аналіз проблематики завдання та існуючих рішень, розробка моделей вирішення завдання, аналіз і вибір засобів та інструментів розробки, розробка програмного проєкту, висновки за результатами роботи.

5. Орієнтовний перелік графічного (ілюстративного) матеріалу

1. Структура проєкту 2. Засоби розробки 3. Розробка програмного продукту 4. Взаємодія користувача з додатком

6. Дата видачі завдання 13 вересня 2025 р.

Календарний план

№ з/п	Назва етапів виконання дипломної роботи	Термін виконання етапів дипломної роботи	Примітка
1	Затвердження теми роботи		
2	Вивчення та аналіз задачі	14-18.04.2025	
3	Розробка архітектури та загальної структури системи	20.04-24.04.2025	
4	Розробка структур окремих підсистем	24.04-01.05.2025	
5	Програмна реалізація системи	02-06.05.2025	
6	Оформлення пояснювальної записки	07-11.05.2025	
7	Захист програмного продукту	12-16.05.2025	
8	Передзахист	26-28.05.2025	
9	Подання готової роботи на кафедру	06.06.2025	
10	Захист	16-20.06.2025	

Студент _____ **Кирил КУЧИНСЬКИЙ**
(підпис) (ім'я, ПРІЗВИЩЕ)

Керівник роботи _____ **Антон ПАСІЧНЮК**
(підпис) (ім'я, ПРІЗВИЩЕ)

АНОТАЦІЯ

Дипломна робота складається з 61 сторінки, включає 16 ілюстрацій, 1 додаток та 20 джерел у списку використаної літератури.

Метою даної роботи було створення веб-додатку для децентралізованого голосування на основі блокчейн-технологій.

Створений додаток забезпечує прозоре та безпечне голосування, дозволяючи створювати опитування, авторизувати виборців, фіксувати голоси в блокчейні Ethereum та відображати статистику в реальному часі через інтуїтивний інтерфейс. Додаток підтримує автентифікацію користувачів через MetaMask, управління адміністраторами та відображення результатів у вигляді графіків і таблиць.

Для написання програмного продукту було обрано мову Solidity для смарт-контракту, фреймворк Nest.js із TypeScript для серверної частини та React.js із TypeScript для клієнтської частини. У якості середовищ розробки використано IntelliJ IDEA, бібліотеку Hardhat для тестування смарт-контракту та набір інструментів Foundry Anvil для локального розгортання блокчейну. Для створення користувацького інтерфейсу використано бібліотеку Material UI, а для взаємодії з блокчейном — Ethers.js. Тестові дані генерувалися за допомогою Faker.js.

Ключові слова: блокчейн, децентралізоване голосування, смарт-контракт, Solidity, Ethereum, JavaScript, MetaMask, Ethers.js.

ABSTRACT

The thesis consists of 61 pages, includes 16 illustrations, 1 appendix, and 20 sources in the list of references.

The objective of this thesis was to develop a web application for decentralized voting based on blockchain technologies.

The developed application ensures transparent and secure voting, enabling the creation of polls, voter authorization, recording votes on the Ethereum blockchain, and real-time display of statistics through an intuitive interface. The application supports user authentication via MetaMask, administrator management, and the presentation of results in the form of charts and tables.

For the development of the software product, the Solidity language was chosen for the smart contract, the Nest.js framework with TypeScript for the server side, and React.js with TypeScript for the client side. The development environments included IntelliJ IDEA, the Hardhat library for smart contract testing, and the Foundry Anvil toolkit for local blockchain deployment. The Material UI library was used for creating the user interface, Ethers.js for blockchain interaction, and Faker.js for generating test data.

Keywords: blockchain, decentralized voting, smart contract, Solidity, Ethereum, JavaScript, MetaMask, Ethers.js.

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ, СКОРОЧЕНЬ І ТЕРМІНІВ 9
ВСТУП 10
1 ЗАДАЧА РОЗРОБКИ ДОДАТКУ ДЕЦЕНТРАЛІЗОВАНОГО ГОЛОСУВАННЯ . 11
1.1 Аналіз призначення додатку для децентралізованого голосування та потенційні користувачі 11
1.2 Задачі додатку для децентралізованого голосування 12
1.3 Структура системи додатку для децентралізованого голосування 12
1.3.1 Роль смарт-контракту 13
1.3.2 Призначення серверної частини 13
1.3.3 Призначення клієнтської веб частини 14
2 АНАЛІЗ ПРОБЛЕМИ ДЕЦЕНТРАЛІЗОВАНОГО ГОЛОСУВАННЯ 15
2.1 Визначення та ключові характеристики централізованого голосування 15
2.2 Вразливості та недоліки централізованих систем голосування 16
2.3 Децентралізоване голосування на основі блокчейн технологій 17
2.4 Порівняльний аналіз централізованого та децентралізованого голосування ... 19
2.5 Огляд існуючих рішень 19
2.5.1 Централізовані системи голосування 20
2.5.2 Децентралізовані системи голосування 20
2.6 Проблеми та виклики реалізації системи голосування 21
3 ЗАСОБИ РОЗРОБКИ ДОДАТКУ 22
3.1 Розподілена система блокчейн 22
3.1.1 Основні компоненти блокчейн-системи 22
3.1.2 Принцип роботи блокчейн-системи 23
3.2 Смарт-контракти 24
3.2.1 Мова програмування Solidity 25
3.2.2 Середовище Ethereum Virtual Machine 25
3.2.3 Середовища розробки смарт-контрактів Hardhat та Foundry 26
3.3 Мови програмування JavaScript та TypeScript 27

3.4 Клієнтська частина 27
3.4.1 Бібліотека React.js 28
3.4.2 Бібліотека компонентів інтерфейсу Material UI 29
3.4.3 Бібліотека Ethers.js та розширення для браузерів MetaMask 30
3.5 Серверна частина 31
3.5.1 Серверний фреймворк Nest.js 32
3.5.2 Інжекція залежностей на прикладі Nest.js 32
3.5.3 Генерація даних за допомогою бібліотеки Faker.js 34
4 ОПИС ПРОГРАМНОЇ РЕАЛІЗАЦІЇ ДОДАТКУ 35
4.1 Структура смарт-контракту додатку 36
4.1.1 Основні компоненти контракту 36
4.1.2 Функціонал контракту 38
4.1.3 Тестування смарт-контракту 38
4.2 Серверна частина 41
4.2.1 Модуль взаємодії з блокчейном 42
4.2.2 Допоміжний модуль 42
4.2.3 Модуль статистики 43
4.3 Клієнтська частина 43
4.3.1 Вкладка статистики 43
4.3.2 Вкладка голосування 44
4.3.3 Адмін-панель 45
5 ВЗАЄМОДІЯ КОРИСТУВАЧА З ДОДАТКОМ 46
5.1 Системні вимоги та налаштування 46
5.1.1 Веб-браузер 47
5.1.2 Додаток MetaMask 47
5.2 Перегляд статистики 47
5.3 Автентифікація користувачів 49
5.4 Голосування користувачем 50
5.5 Адміністрування 52
ВИСНОВКИ 53

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ 54
ДОДАТОК А 56

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ, СКОРОЧЕНЬ І ТЕРМІНІВ

Блокчейн — технологія розподіленого реєстру, що забезпечує незмінність і прозорість даних шляхом зберігання транзакцій у зв’язаних блоках

Смарт-котнракт — набір правил та комп’ютерних алгоритмів для автоматичного контролю виконання угод

Ethereum — децентралізований блокчейн для виконання смарт-контрактів

EVM — Ethereum Virtual Machine, віртуальна машина для виконання смарт-контрактів у мережі Ethereum

Solidity — контрактно-орієнтована мова програмування для створення смарт-контрактів на Ethereum

MetaMask — криптогаманець і розширення для браузера, що дозволяє користувачам взаємодіяти з блокчейном Ethereum

JavaScript — високорівнева, об’єктно орієнтована прототипна мова програмування

TypeScript — надбудова над JavaScript, що додає статичну типізацію

Hardhat — середовище розробки для компіляції, тестування та розгортання смарт-контрактів на Ethereum

Foundry Anvil — інструмент для створення локальної тестової мережі Ethereum

HTTP — Hypertext Transfer Protocol, протокол обміну даними

RPC — Remote Procedure Call, протокол виклику процедур в віддалених системах

API — Application Programming Interface, набір правил для взаємодії з програмою

JSON — JavaScript Object Notation, формат обміну даних

ВСТУП

Сучасні виборчі процеси потребують високого рівня довіри, прозорості та безпеки, що є ключовими викликами для традиційних систем голосування. Проблематика голосувань полягає у забезпеченні відкритості підрахунку голосів, захисті від зовнішнього втручання та створенні умов для рівного доступу всіх учасників. Традиційні централізовані системи часто не відповідають цим вимогам через залежність від посередників, що знижує довіру до результатів. Блокчейн-технології відкривають нові можливості для вирішення цих проблем завдяки децентралізації, незмінності даних і прозорості транзакцій.

Робота ставить за мету створення веб-додатку для децентралізованого голосування на основі блокчейн-технологій, який забезпечує прозоре управління опитуваннями, безпечне голосування та відкритий доступ до результатів. Розроблений додаток дозволяє користувачам автентифікуватися через MetaMask, створювати опитування, голосувати та переглядати статистику в реальному часі. Використання смарт-контрактів на Solidity гарантує надійність і прозорість обробки голосів, а децентралізована архітектура виключає потребу в централізованих посередниках.

На сьогодні аналіз ринку підтверджує ріст сфери блокчейн-технологій. Вони захоплюють все більшу частину ринку та підвищують надійність використання у виборчих системах, зокрема через їх здатність забезпечувати криптографічний захист і публічну верифікацію. Розроблений додаток спирається на ці досягнення, пропонуючи зручний інтерфейс і функціонал для адміністраторів і виборців.

1 ЗАДАЧА РОЗРОБКИ ДОДАТКУ ДЕЦЕНТРАЛІЗОВАНОГО ГОЛОСУВАННЯ

Створення надійної та прозорої системи голосування є складним процесом. Метою розробки є створення веб-додатку для децентралізованого голосування на основі блокчейн-технологій, який забезпечує безпеку та доступність виборчих процесів. Задача полягає в реалізації системи, що дозволяє створювати опитування, авторизувати виборців, проводити голосування та відображати результати в реальному часі. Розробка додатку має на меті вирішення проблем традиційних і частково децентралізованих систем голосування, таких як непрозорість, вразливість до маніпуляцій і залежність від централізованих посередників [1].

1.1 Аналіз призначення додатку для децентралізованого голосування та потенційні користувачі

Додаток призначений для організації прозорих і безпечних голосувань у децентралізованій мережі Ethereum. Він має забезпечувати створення опитувань, автентифікацію користувачів за допомогою MetaMask, голосування та відображення статистики. Цільові користувачі включають:

– власника контракту, який розгортає смарт-контракт і управляє адміністраторами;

– адміністраторів, які створюють опитування, авторизують виборців і завершують опитування;

– виборців, які голосують за опції в опитуваннях, авторизуючись в додатку за допомогою MetaMask.

Додаток має підтримувати різні сценарії використання, від локальних опитувань до масштабних виборів, забезпечуючи гнучкість і доступність.

1.2 Задачі додатку для децентралізованого голосування

Додаток ставить перед собою розв’язання задач, пов’язаних з організацією голосувань, забезпечуючи прозорість, безпеку та зручність. Основні задачі:

– створення та управління опитуваннями;

– авторизація виборців для участі в голосуванні;

– проведення голосування з фіксацією голосів у блокчейні;

– відображення статистики та результатів у реальному часі.

Вхідні дані включають назви опитувань, терміни, опції та адреси виборців, а вихідні — результати голосування, статистичні графіки та таблиці.

1.3 Структура системи додатку для децентралізованого голосування

Система має складатись з трьох компонентів, які взаємодіють для забезпечення повного циклу голосування (рисунок 1.1).

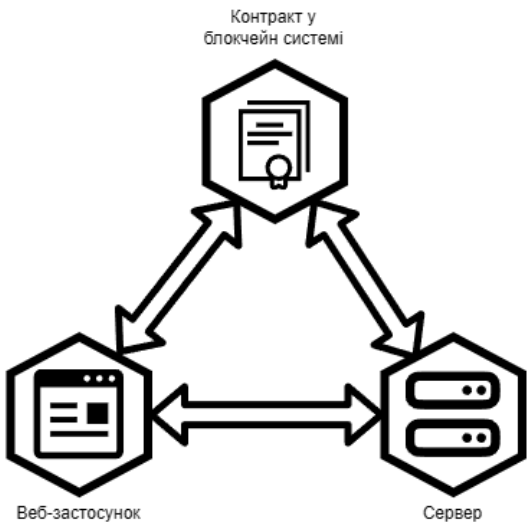

Рисунок 1.1 – Схема архітектури системи

Таке розподілення обов’язків та задач в системі створює мінімальне необхідне надійне та зручне функціонування.

1.3.1 Роль смарт-контракту

Основною задачею для смарт-контракту є створення надійного механізму для управління процесом голосування, включаючи створення опитувань, авторизацію виборців, фіксацію голосів і визначення результатів. Смарт-контракт має забезпечити незмінність даних, публічну верифікацію транзакцій і захист від маніпуляцій, що вирішує проблему недовіри до централізованих систем.

Смарт-контракт є автономним компонентом, розгорнутим у блокчейн-мережі, яка підтримує виконання програмного коду, наприклад, у мережі з віртуальною машиною, сумісною з децентралізованими протоколами.

Порівняно з альтернативами, такими як централізовані бази даних, смарт-контракт усуває потребу в довіреному посереднику, але вимагає ретельного тестування через незмінність після розгортання.

1.3.2 Призначення серверної частини

Серверна частина додатку має забезпечити ефективну взаємодію між клієнтським інтерфейсом і смарт-контрактом, а також обробку даних для аналітики та тестування. Задача полягає в подоланні складнощів інтеграції з блокчейном, таких як асинхронна обробка транзакцій, агрегація великих обсягів даних і генерація тестових сценаріїв.

Один з кращих варіантів серверної архітектури є модульна система, що складається з компонентів для взаємодії з блокчейном, обробки статистики та генерації тестових даних. Модульність дозволяє ізолювати логіку, зменшуючи ризик конфліктів між різними функціями, і спрощує майбутнє масштабування, наприклад, додавання нових аналітичних функцій.

Порівняно з альтернативними підходами, такими як монолітна серверна архітектура, модульний дизайн підвищує гнучкість, але може вимагати більше часу на початкове налаштування. Використання безсерверної-архітектури може зменшити витрати на інфраструктуру, але ускладнює обробку складних транзакцій із блокчейном через обмеження на час виконання. Перевагою модульного підходу є можливість швидкого доповнення функціоналу, наприклад, нових точок виклику для аналітики, а також ефективна обробка даних завдяки чіткому розподілу обов'язків між модулями.

1.3.3 Призначення клієнтської веб частини

Клієнтська веб-частина має забезпечити інтуїтивний і зручний інтерфейс для користувачів, включаючи виборців, адміністраторів і власника системи, з підтримкою автентифікації через криптогаманець і відображенням статистики в реальному часі. Задача полягає в створенні динамічного інтерфейсу, який мінімізує бар'єри для користувачів без технічних знань, і водночас інтегрується з блокчейном для виконання транзакцій.

Архітектурно клієнтська частина має бути розроблена як односторінковий додаток із компонентним підходом, що включає три основні вкладки: головну сторінку (дешборд) для перегляду статистики, вкладку голосування для взаємодії з опитуваннями та адмін-панель для управління системою. Інтерфейс має інтегруватись з криптогаманцем, що дозволить користувачам автентифікуватися та підписувати транзакції. Порівняно з альтернативами, такими як традиційні багатосторінкові додатки, односторінковий варіант забезпечує швидший відгук інтерфейсу, але вимагає стабільного Інтернет-з'єднання.

Мобільний додаток міг би покращити продуктивність на смартфонах, але значно збільшує витрати на розробку та підтримку. Перевагою веб варіанту є швидка розробка, адаптивність для різних пристроїв і безпечна автентифікація через гаманець, що робить систему доступною для широкої аудиторії.

2 АНАЛІЗ ПРОБЛЕМИ ДЕЦЕНТРАЛІЗОВАНОГО ГОЛОСУВАННЯ

Еволюція демократичних процесів нерозривно пов'язана з методами голосування, які використовуються для вираження волі громадян. Традиційно ці системи були централізованими, покладаючись на довіру до єдиного органу або обмеженої кількості суб'єктів. Однак поява технологій розподіленого реєстру, зокрема блокчейну, відкрила шлях до децентралізованих систем голосування, які обіцяють підвищити безпеку, прозорість та доступність.

2.1 Визначення та ключові характеристики централізованого голосування

Централізовані системи голосування визначаються тим, що єдиний, авторитетний суб'єкт або обмежена кількість суб'єктів керують і контролюють весь виборчий процес. Це включає програмування виборів, створення бюлетенів, сканування голосів, підрахунок та загальний нагляд за системою. Типово ці системи складаються з системи управління виборами, систем підрахунку голосів на дільницях, центральних систем підрахунку та рішень для доступного голосування [2].

Довіра до централізованих систем покладається на ці центральні органи та їхнє дотримання суворих процедур. Ці процедури включають процеси "довіреної збірки", що забезпечують безпечний та задокументований ланцюжок постачання для програмного забезпечення та обладнання. Критичні компоненти часто ізольовані від інтернету для захисту від кіберзагроз, а доступ до системи вимагає багатофакторної аутентифікації [2]. Навіть у деяких "розподілених" архітектурах електронного голосування, центральний виборчий орган часто контролює місцеві органи, зберігаючи певний ступінь централізованого контролю [3].

Основна філософія централізованого голосування, з її акцентом на "довіреній збірці", "закритому ланцюжку постачання" та "ізольованих" системах, принципово базується на локалізації та контролі в межах фізично та процедурно захищеного периметра [2]. Цей підхід різко контрастує з моделлю розподіленої довіри блокчейну, де безпека походить від мережевого консенсусу, а не від єдиної точки контролю. Така конструкція передбачає, що якщо середовище та процес можуть бути повністю контрольовані, то цілісність може бути гарантована.

2.2 Вразливості та недоліки централізованих систем голосування

Незважаючи на ретельні процедури, централізовані системи голосування мають значні вразливості, які можуть підірвати їхню цілісність та суспільну довіру. Ці системи схильні до маніпуляцій, їм бракує перевірених аудиторських слідів, і вони часто виключають віддалених виборців [4].

Однією з основних проблем є ризики фізичного втручання. Електронні машини для голосування (EVM) вразливі до фізичного втручання під час транспортування та зберігання. Це створює можливість для зловмисників отримати доступ до обладнання та потенційно змінити його до або під час виборів [4].

Крім того, вразливості централізованих баз даних становлять серйозну кіберзагрозу. Залежність від централізованих баз даних робить їх схильними до різних кібератак, включаючи атаки типу "відмова в обслуговуванні" та атаки "людина посередині". Ці атаки можуть перешкоджати нормальній роботі системи, запобігати реєстрації голосів або компрометувати цілісність даних [5].

Централізованим системам часто бракує криптографічного підтвердження цілісності голосів, а їхні процеси підрахунку результатів можуть бути непрозорими. Це вимагає сліпої довіри до центральних органів і унеможливлює незалежну перевірку виборцями своїх зареєстрованих голосів, що призводить до браку прозорості та обмежених можливостей аудиту [4].

З погляду операційної ефективності, централізовані системи несуть високі витрати на персонал та фізичну інфраструктуру, страждають від затримок через ручний підрахунок, а паперові системи верифікації сприяють негативному впливу на навколишнє середовище [5].

2.3 Децентралізоване голосування на основі блокчейн технологій

Децентралізоване голосування, що базується на блокчейні, є відповіддю на недоліки централізованих систем, пропонуючи новий підхід до забезпечення цілісності та довіри до виборчих процесів. Ця парадигма використовує властивості блокчейну для розподілу контролю та підвищення прозорості [6]. Приклад архітектури системи голосування на основі блокчейну наведену в рисунку 2.1 [5].

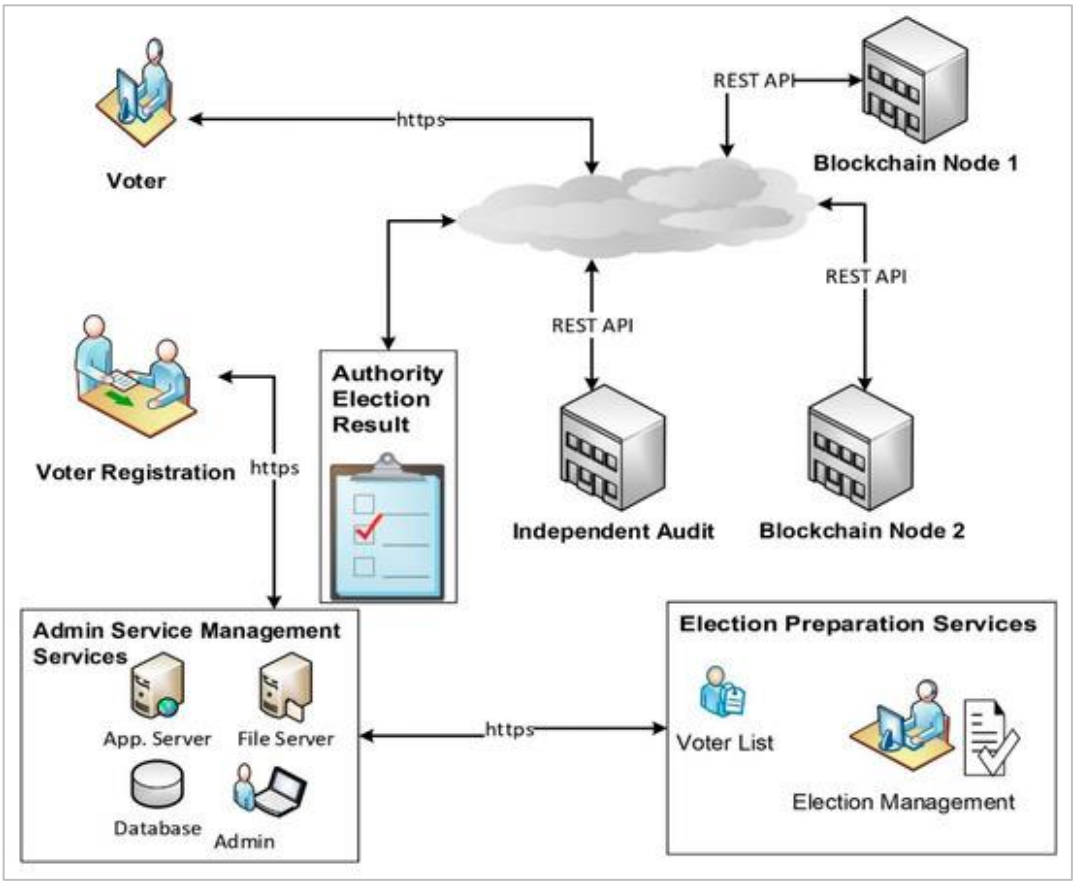

Рисунок 2.1 — Архітектура блокчейн системи для голосування

Децентралізовані системи голосування розподіляють повноваження та контроль між багатьма учасниками, а не зосереджують їх в одній організації. Вони використовують такі технології, як блокчейн, для забезпечення прозорості, безпеки та надійності. У блокчейн-системі кожен учасник (вузол) має копію реєстру голосування, який записує всі голоси. Цей розподілений реєстр оновлюється та підтримується численними мережевими вузлами, запобігаючи повний контроль єдиної організації над системою [6].

Основні властивості блокчейну — децентралізація, прозорість та незмінність — використовуються для забезпечення безпечних, захищених від втручання та перевірених виборчих процесів. Смарт-контракти відіграють ключову роль, автоматизуючи робочі процеси виборів, керуючи подачею голосів та забезпечуючи незмінність записів. Вони є самовиконуваними програмами в блокчейні, які забезпечують виконання правил без посередників. Віртуальна машина Ethereum (EVM), як основний механізм для Ethereum, виконує код смарт-контрактів детерміновано та незмінно на всіх вузлах мережі, забезпечуючи послідовність та безпеку [6].

Переваги децентралізованого голосування на основі блокчейну:

– підвищена безпека та захист від втручання;

– підвищена прозорість та можливість аудиту;

– усунення посередників та автоматизація;

– підвищена доступність та участь;

– стійкість та опір цензурі;

– довіра та легітимність.

Перехід від фізичної безпеки та процедурної довіри в централізованих системах до криптографічної безпеки та перевіреної прозорості в децентралізованих системах є фундаментальною зміною парадигми в тому, як досягається цілісність виборів.

2.4 Порівняльний аналіз централізованого та децентралізованого голосування

Порівняння централізованих та децентралізованих систем голосування виявляє їхні фундаментальні відмінності в архітектурі, принципах роботи та підходах до вирішення ключових виборчих завдань.

Централізовані системи концентрують контроль в одній або кількох організаціях, таких як виборчі органи або конкретні сервери. Це означає, що довіра до системи покладається на ці органи та їхню здатність діяти чесно та безпечно. Навпаки, децентралізовані системи розподіляють контроль по мережі вузлів, усуваючи єдину точку відмови та зменшуючи залежність від центрального органу. Це призводить до більш стійкої та нецензурованої системи.

Централізовані системи покладаються на централізовані бази даних, які можуть бути вразливими до злому або маніпуляцій. Децентралізовані системи використовують розподілений реєстр (блокчейн), де кожен учасник має копію, забезпечуючи незмінність та прозорість. Кожна транзакція криптографічно пов'язана з попередньою, створюючи незмінний ланцюжок записів.

Централізовані системи вимагають довіри до інституцій та їхніх фізичних/процедурних засобів контролю, тоді як децентралізовані системи переносять довіру на криптографічні докази та розподілений мережевий консенсус. Це означає філософську розбіжність у тому, як досягається та перевіряється цілісність виборів.

2.5 Огляд існуючих рішень

Сучасні системи голосування поділяються на централізовані та децентралізовані. Кожен тип має свої особливості, які впливають на їх ефективність і довіру з боку користувачів. Аналіз цих рішень дозволяє виявити недоліки, які вирішує розроблений додаток.

2.5.1 Централізовані системи голосування

Централізовані системи голосування, такі як Dominion Voting Systems або традиційні електронні системи, використовуються в багатьох країнах для виборів. Вони базуються на серверах, керованих організаторами голосування, і включають бази даних для зберігання голосів.

Переваги:

– простота впровадження та масштабування;

– зручність адміністрування для організаторів;

– низька вартість порівняно з блокчейн-системами.

Недоліки:

– непрозорість — користувачі не мають доступу до даних, що ускладнює верифікацію результатів;

– вразливість до маніпуляцій — адміністратори серверів можуть змінювати результати, як це було підозрювано в деяких виборах;

– залежність від посередників — довіра до системи залежить від чесності організаторів.

Наприклад, у Dominion Voting Systems голоси зберігаються на централізованих серверах, що створює ризик несанкціонованого доступу. Розроблений додаток усуває ці проблеми, використовуючи блокчейн для забезпечення незмінності даних і публічної верифікації.

2.5.2 Децентралізовані системи голосування

Децентралізовані системи, такі як Voatz, Follow My Vote і Polys, використовують блокчейн для підвищення прозорості та безпеки. Вони дозволяють фіксувати голоси в розподіленому реєстрі, що ускладнює маніпуляції [7].

Voatz це мобільний додаток для голосування, який використовує блокчейн для фіксації голосів і біометричну автентифікацію. Він використовує зручний інтерфейс, підтримує мобільні пристрої. З недоліків можна зауважити залежність від централізованих серверів для автентифікації, що знижує децентралізацію. Для

вирішення цієї проблеми розроблений додаток використовує MetaMask для автентифікації, що спрощує доступ і усуває потребу в централізованих серверах.

Блокчейн-платформа для голосування Follow My Vote з відкритим кодом, яка використовує власний блокчейн, має високу прозорість завдяки публічному реєстру та можливості аудиту. З недоліків є складна верифікація особи через сторонні сервіси, що знижує анонімність. В порівняння розроблений додаток працює з Ethereum, що забезпечує сумісність і доступність. Автентифікація через MetaMask зберігає анонімність виборців.

2.6 Проблеми та виклики реалізації системи голосування

Реалізація децентралізованого голосування стикається з низкою технічних і організаційних викликів. Аналіз цих проблем дозволяє визначити вимоги до розробки та обґрунтувати вибір технологій.

Безпека смарт-контрактів і автентифікація користувачів є критичними для довіри до системи. Основні виклики:

– атаки на смарт-контракти та вразливості, такі як reentrancy або overflow, можуть дозволити зловмисникам маніпулювати голосами;

– автентифікація забезпечення доступу лише авторизованим виборцям без порушення анонімності;

– захист приватних ключів користувачів, які використовуються MetaMask для автентифікації в додатку.

Для подолання цих проблем було використано перевірки та ліміти на запити до системи, проведено тестування та оптимізацію функцій смарт-контракту. Це зменшує виниклі ризики та будує надійну архітектуру для голосування.

3 ЗАСОБИ РОЗРОБКИ ДОДАТКУ

Розробка веб-додатку для децентралізованого голосування на основі блокчейн-технологій вимагала використання сучасних програмних середовищ, технологій і бібліотек, які забезпечують надійність, безпеку та зручність реалізації.

3.1 Розподілена система блокчейн

Блокчейн – це розподілена база даних, яка забезпечує збереження та передачу інформації у вигляді послідовного ланцюга блоків. Кожен блок містить набір транзакцій, хеш попереднього блоку та власний хеш, що гарантує цілісність та незмінність даних. Завдяки децентралізованій структурі та використанню криптографічних методів, блокчейн забезпечує високий рівень безпеки та прозорості [8].

3.1.1 Основні компоненти блокчейн-системи

Блокчейн загалом складається з декількох складових:

– блок — структура, що містить набір транзакцій, мітку часу, хеш попереднього блоку та власний хеш;

– транзакція — операція передачі даних або цінностей між учасниками мережі;

– вузли мережі (ноди) — комп’ютери, що підтримують роботу мережі, зберігають копії блокчейну та беруть участь у верифікації транзакцій;

– механізм консенсусу — алгоритм, за допомогою якого вузли мережі досягають згоди щодо додавання нових блоків до ланцюга;

– смарт-контракти — програмні коди, що автоматично виконують умови договорів при настанні визначених подій.

3.1.2 Принцип роботи блокчейн-системи

Загалом весь процес транзакції в блокчейні можна описати в 5 кроків:

1. Ініціація транзакції: користувач створює транзакцію, яка підписується його приватним ключем.

2. Розповсюдження транзакції: транзакція передається до всіх узлів мережі для перевірки.

3. Верифікація транзакції: узли перевіряють автентичність транзакції та її відповідність правилам мережі.

4. Формування блоку: після верифікації транзакції об’єднуються у блок, який додається до ланцюга після досягнення консенсусу.

5. Оновлення блокчейну: новий блок розповсюджується по мережі, і кожен узол оновлює свою копію блокчейну.

Приклад обробки транзакцій зображено на рисунку 3.1.

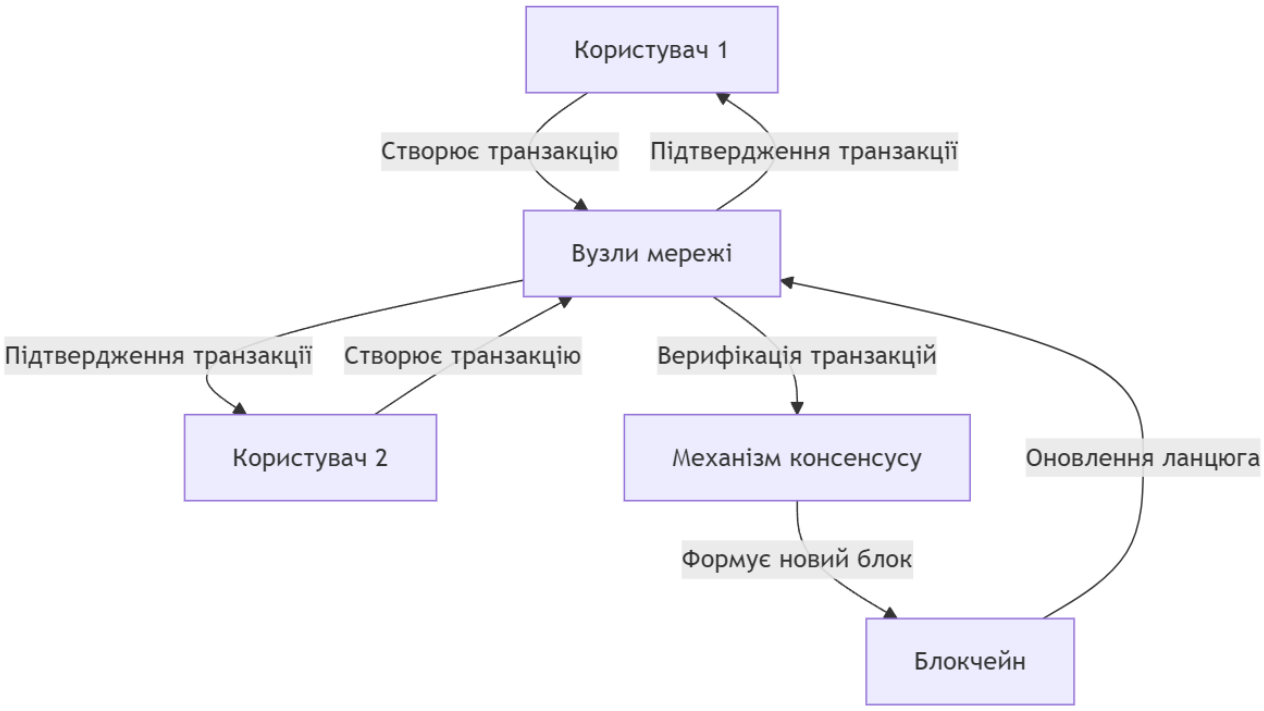

Рисунок 3.1 — Приклад обробки транзакцій в блокчейні

Таким чином блокчейн-технологія забезпечує надійний та прозорий спосіб зберігання і передачі інформації без потреби в централізованому управлінні.

3.2 Смарт-контракти

Смарт-контракт — це програмний код, що автоматично виконує умови договору, зафіксованого у ньому. Смарт-контракти зберігаються у блокчейні та запускаються автоматично після виконання заздалегідь визначених умов. Головною перевагою смарт-контрактів є те, що вони виключають потребу в посередниках, оскільки всі умови та наслідки взаємодій виконуються прозоро та неупереджено [9].

Основні властивості смарт-контрактів:

– детермінованість — результат виконання контракту завжди однаковий при однаковому наборі вхідних даних;

– автономність — контракт виконується без зовнішнього втручання;

– незмінність — після розгортання контракту в мережі змінити його код неможливо;

– прозорість — усі учасники мережі можуть переглядати умови контракту.

Для розробки смарт-контракту обрано Solidity, яка є основною мовою програмування для Ethereum. Альтернативами могли бути Vyper, ще одна мова для Ethereum або Rust, яка використовується для блокчейну Solana. Solidity була обрана через її широку популярність, велику екосистему та підтримку складної логіки, необхідної для реалізації функцій голосування, авторизації та фіналізації.

Vyper, хоча й пропонує простіший синтаксис і підвищену безпеку завдяки обмеженню деяких функцій (наприклад, відсутності рекурсії), менш гнучка для складних структур, таких як мапінги та події, які активно використовуються в додатку. Rust, який застосовується в Solana, забезпечує високу продуктивність, але Solana має менш розвинуту екосистему для децентралізованих додатків порівняно з Ethereum.

Переваги Solidity включають велику кількість документації, бібліотек (наприклад, OpenZeppelin) і спільноту розробників, що полегшило написання та тестування контракту. Крім того, сумісність із Ethereum Virtual Machine (EVM) забезпечує можливість розгортання в різних мережах.

3.2.1 Мова програмування Solidity

Мова програмування Solidity — це високорівнева об’єктно-орієнтована мова програмування, спеціально створена для написання смарт-контрактів на платформі Ethereum. Ця мова має статичну типізацію, підтримує спадкування, бібліотеки і складні структури даних. Також в ній використовуються спеціальні типи для адрес, токенів та управління грошима. Для полегшення відстежування подій в блокчейні Solidity має можливість визначення подій, що викликаються під час виконання контракту, які вони подібні до логування.

Смарт-контракт на Solidity компілюється у байткод, який виконується на Ethereum Virtual Machine.

3.2.2 Середовище Ethereum Virtual Machine

Ethereum Virtual Machine (EVM) — це середовище виконання, що забезпечує запуск смарт-контрактів на всіх вузлах мережі Ethereum. EVM інтерпретує байткод контракту, створеного компілятором Solidity, і гарантує однакове виконання на кожному з вузлів мережі.

Основні функції EVM:

– виконання транзакцій та смарт-контрактів;

– контроль ресурсів через механізм Gas (обмеження обчислювальної потужності);

– ізоляція контрактів, що запобігає доступу одного контракту до пам’яті іншого;

– універсальність — EVM підтримується не лише Ethereum, а й багатьма сумісними блокчейн-системами (наприклад, BNB Smart Chain, Polygon).

Модель обчислення EVM заснована на стековій архітектурі та має власний набір інструкцій, подібний до байт-коду віртуальної машини Java.

3.2.3 Середовища розробки смарт-контрактів Hardhat та Foundry

У розробці децентралізованих застосунків важливо мати надійне середовище для створення, тестування та розгортання смарт-контрактів. Серед найпоширеніших інструментів у цій сфері є Hardhat та Foundry, які істотно спрощують процеси розробки на базі мереж Ethereum та EVM-сумісних блокчейнів.

Hardhat — це середовище розробки смарт-контрактів на мові Solidity, написане на JavaScript/TypeScript. Воно надає гнучку інфраструктуру для компіляції, тестування, налагодження та розгортання контрактів.

Основні можливості Hardhat:

– локальна мережа Hardhat Network, що дозволяє миттєво тестувати контракти без потреби у зовнішній мережі;

– розширення через плагіни від інших розробників;

– інтеграція з Ethers.js для зручної роботи з контрактами у тестах та скриптах;

– конфігурація через окремий файл, що дозволяє гнучко керувати середовищами, мережами, ключами та параметрами компіляції;

– підтримка Solidity Debugging — можливість налагодження коду смарт-контракту за допомогою консольного логування;

– сумісність з MetaMask та основними тестовими мережами (Goerli, Sepolia, Polygon, BSC тощо).

Foundry — це набір швидких інструментів для роботи зі смарт-контрактами, написаних на мові Rust. Foundry особливо цінується за свою продуктивність, інтеграцію з Solidity та підтримку розробки без Node.js. Цей набір зазвичай використовують для тестування в більш навантажених сценаріях.

Foundry складається з декількох інструментів:

– forge — основний інструмент для компіляції, тестування, розгортки та управління контрактами;

– cast — інструмент для взаємодії з мережами Ethereum (виклик функцій контракту, підпис транзакцій, зчитування стану тощо);

– anvil — локальна Ethereum-сумісна мережа (аналог Hardhat Network), яка дозволяє емулювати блокчейн у тестовому середовищі.

3.3 Мови програмування JavaScript та TypeScript

JavaScript (JS) — це високорівнева динамічна мова програмування, яка є стандартом для веброзробки. Вона широко використовується для створення інтерактивного інтерфейсу користувача у браузері. JavaScript підтримується всіма сучасними веббраузерами та має великий набір бібліотек і фреймворків, що забезпечують гнучкість та масштабованість вебзастосунків.

TypeScript — це надмножина JavaScript, розроблена компанією Microsoft. Вона додає статичну типізацію та об’єктно-орієнтовані конструкції до JS-коду. TypeScript компілюється у чистий JavaScript, що забезпечує сумісність із будь-яким середовищем виконання JS [10].

Основні переваги TypeScript:

– статична типізація, що зменшує кількість помилок на етапі розробки;

– покращена підтримка IDE, включно з автодоповненням, підсвіткою помилок та інтроспекцією коду;

– можливість використання сучасних можливостей JS незалежно від підтримки браузером.

3.4 Клієнтська частина

Сучасна розробка веб-додатків вимагає використання широкого спектру бібліотек. Клієнтську частину розроблено на React.js із бібліотекою Material UI, які порівнювалися з Vue.js і Angular.

React.js був обраний за його компонентний підхід, високу продуктивність завдяки віртуальному DOM і велику спільноту, що полегшує інтеграцію з бібліотеками, такими як Chart.js для графіків і Ethers.js для взаємодії з блокчейном. Vue.js, хоча й легший у навчанні, має меншу екосистему бібліотек, що могло ускладнити створення складних компонентів, таких як дешборд із графіками.

Фреймворк Angular, хоча й потужний для великих додатків, має вищий поріг входу та більшу складність для швидкої розробки. Бібліотека компонентів Material UI була обрана за готові адаптивні компоненти (таблиці, форми, кнопки), які забезпечили єдиний стиль інтерфейсу та зекономили час порівняно з Bootstrap чи Ant Design. Переваги React.js включають швидке оновлення інтерфейсу через хуки (useState, useEffect) і легку інтеграцію з MetaMask через Web3 API, а Material UI забезпечила зручність для користувачів на різних пристроях.

Для взаємодії зі смарт-контрактом використано Ethers.js, який можна порівняти з Web3.js. Ethers.js був обраний за його легший і сучасніший API, підтримку TypeScript і кращу сумісність із Nest.js і React.js.

Web3.js, хоча й широко використовується, має складніший синтаксис і більший розмір бібліотеки, що може уповільнити виконання. Ethers.js спростив виклик функцій контракту і обробку подій, що критично для реального часу в дешборді. Переваги Ethers.js включають компактність, швидкість і легке управління транзакціями, що полегшило інтеграцію між фронтендом, бекендом і MetaMask.

3.4.1 Бібліотека React.js

React.js — це JavaScript-бібліотека для побудови інтерфейсів користувача, розроблена Facebook. React дозволяє створювати багаторазові компоненти, які динамічно оновлюються при зміні стану додатка [11].

Ключові характеристики React:

– компонентна архітектура, що сприяє повторному використанню коду;

– віртуальне дерево розмітки, яке оптимізує оновлення інтерфейсу;

– односпрямований потік даних, що забезпечує передбачуваність поведінки додатка.

React є одним із найпопулярніших інструментів для побудови frontend-частини децентралізованих застосунків (dApps).

3.4.2 Бібліотека компонентів інтерфейсу Material UI

Material UI (MUI) — це популярна бібліотека компонентів інтерфейсу для React, яка реалізує концепції дизайну Material Design від Google [12]. Вона дозволяє швидко створювати привабливі, адаптивні та функціональні інтерфейси. Розроблені елементи відповідають сучасним стандартам дизайну.

Приклад бібліотеки можна побачити на рисунку 3.2.

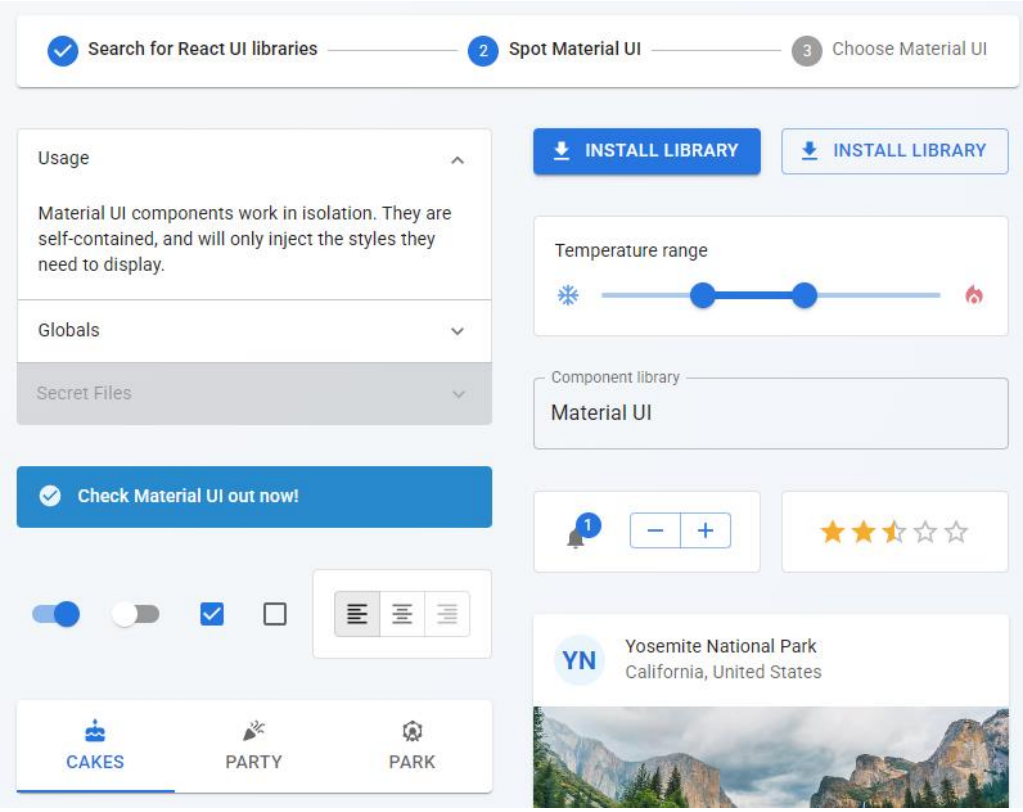

Рисунок 3.2 — Приклад елементів з офіційного веб-сайту бібліотеки Material UI

Переваги використання MUI:

– готові компоненти високої якості (кнопки, форми, діалоги тощо);

– підтримка темізації та кастомізації;

– відповідність вимогам UX/UI-дизайну.

3.4.3 Бібліотека Ethers.js та розширення для браузерів MetaMask

Для звертання до блокчейну та контрактів зазвичай використовують популярну бібліотеку Ethers.js та розширення MetaMask. Виклики до блокчейну здійснюються на основі RPC-через-HTTP протоколу

Ethers.js — це легка та потужна JavaScript/TypeScript-бібліотека для взаємодії з блокчейном Ethereum. Вона забезпечує доступ до смарт-контрактів, облікових записів, підпису транзакцій, читання стану мережі та багато іншого [13].

Основні можливості Ethers.js:

– підключення до провайдерів (такі як MetaMask);

– виконання читання та запису в смарт-контракти (через ABI);

– обробка підписаних повідомлень та транзакцій;

– безпечна робота з приватними ключами (включаючи шифрування та гаманці).

Ethers.js має гарну підтримку та широко використовується в React-додатках для побудови децентралізованого фронтенду.

MetaMask — це браузерне розширення та мобільний додаток, який виконує функцію криптогаманця для мережі Ethereum та сумісних з нею мереж (на основі EVM). MetaMask дозволяє користувачам керувати своїми обліковими записами, підписувати транзакції та взаємодіяти з децентралізованими застосунками (dApp) безпосередньо з браузера [14]. Приклад роботи MetaMask зображено на рисунку 3.3.

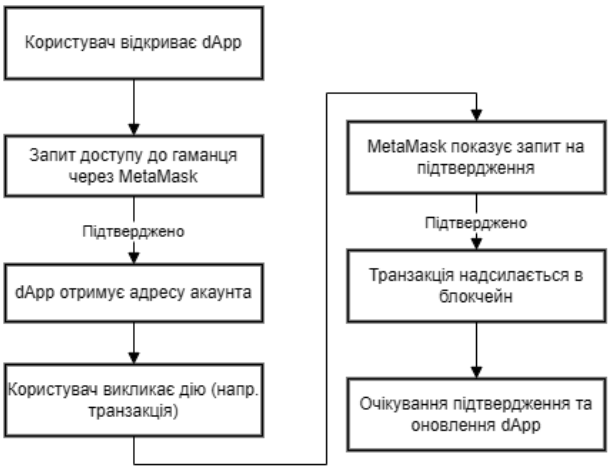

Рисунок 3.3 — Приклад роботи MetaMask з децентралізованим застосунком

Ключові функції MetaMask:

– управління приватними ключами та акаунтами;

– інтеграція з децентралізованими застосунками через стандартизований Web3 API;

– підтвердження транзакцій у зручному графічному інтерфейсі;

– можливість перемикання між мережами (Ethereum, Polygon, Binance Smart Chain тощо).

MetaMask виступає посередником між децентралізованими застосунками та блокчейном, забезпечуючи безпечну та зручну авторизацію користувача.

3.5 Серверна частина

Серверна частина додатку виконує роль посередника між клієнтським додатком і смарт-контрактом, забезпечуючи обробку запитів, генерацію тестових даних і статистичний аналіз. Використано сучасні інструменти для створення модульної та продуктивної серверної частини.

Для серверної частини обрано Nest.js із TypeScript, який порівнювався з Express.js (Node.js) і Django (Python). Nest.js був обраний за його модульну архітектуру, що ідеально відповідає потребам додатку, який має окремі модулі для взаємодії з блокчейном, генерації тестових даних і обробки статистики.

Express.js, хоча й легший і гнучкіший, менш структурований і вимагає додаткових бібліотек для реалізації складної логіки, що збільшує час розробки. Django, популярний для Python-додатків, менш сумісний із JavaScript-екосистемою, що ускладнює інтеграцію з Ethers.js і React.js. TypeScript додав статичну типізацію, що зменшило помилки при обробці даних із смарт-контракту.

Для генерації тестових даних у допоміжному модулі використано Faker.js, який можна порівняти з Mock.js. Faker.js був обраний за його простоту, підтримку широкого спектра даних (назви, адреси, випадкові числа) і сумісність із Node.js, що ідеально для Nest.js. Mock.js, хоча й підходить для створення фіктивних API, менш

гнучкий для генерації даних, специфічних для блокчейну, таких як Ethereum-адреси. Переваги Faker.js включають швидке створення реалістичних даних для ендпоінтів, що прискорило тестування сценаріїв із великою кількістю голосів.

3.5.1 Серверний фреймворк Nest.js

Nest.js — це прогресивний фреймворк для побудови серверних застосунків, який базується на TypeScript. Nest.js поєднує в собі елементи об’єктно-орієнтованого програмування, функціонального програмування та архітектурних шаблонів, таких як ін’єкція залежностей (Dependency Injection) та модульність [15].

Фреймворк широко використовується для побудови REST API, GraphQL API, мікросервісів, WebSockets-сервісів, а також для інтеграції з базами даних, чергами повідомлень, сторонніми сервісами тощо.

Основні характеристики Nest.js:

– модульна архітектура: кожен функціонал інкапсульовано в окремий модуль;

– TypeScript за замовчуванням, з підтримкою чистого JS;

– потужна система інжекції залежностей (Dependency Injection, DI);

– підтримка багатьох популярних шаблонів серверної розробки, наприклад, middleware, interceptor, pipe, guard;

– легка інтеграція з системами об’єктно-реляційного відображення ORM (TypeORM, Prisma, Sequelize, RabbitMQ, Redis тощо).

3.5.2 Інжекція залежностей на прикладі Nest.js

Інжекція залежностей (Dependency Injection) — це ключова концепція Nest.js, яка дозволяє легко керувати залежностями між компонентами за допомогою інверсії керування (Inversion of Control) [16].

Написання коду за допомогою фреймворку зображено на рисунку 3.4.

```
@Injectable()
export class UsersService {
  findAll() {
    return ['user1', 'user2'];
  }
}

@Controller('users')
export class UsersController {
  constructor(private readonly usersService: UsersService) {}

  @Get()
  getUsers() {
    return this.usersService.findAll();
  }
}
```

Рисунок 3.4 — Приклад коду з ін’єкцією залежностей в фреймворку Nest.js

Як це працює:

– компоненти (наприклад, сервіси) оголошуються як інжектовані класи за допомогою декораторів @Injectable();

– коли компонент залежить від іншого — Nest автоматично створює та підставляє інстанси;

– механізм реалізований через власний контейнер залежностей, що створюється при запуску додатка.

Переваги DI:

– висока тестованість (можна легко підмінити залежності у тестах);

– слабке зв’язування компонентів;

– гнучка модульна архітектура;

– зменшення повторюваного шаблонізованого коду;

– простота зміни реалізації;

– масштабованість;

– покращення читабельності та організації коду.

3.5.3 Генерація даних за допомогою бібліотеки Faker.js

Faker.js — це популярна JavaScript -бібліотека, яка дозволяє швидко генерувати реалістичні, але випадкові дані, що часто використовуються для тестування API, демонстраційних додатків та наповнення бази даних [17].

Простий приклад використання бібліотеки можна побачити на рисунку 3.5.

```
import { faker } from '@faker-js/faker';

const user = {
  name: faker.person.fullName(),
  email: faker.internet.email(),
  address: faker.location.streetAddress(),
  phone: faker.phone.number(),
};
```

Рисунок 3.5 — Приклад використання Faker.js для генерації даних користуавча

Основні можливості:

– генерація імен, адрес, email-адрес, телефонів, дат, зображень, валют, випадкових чисел тощо;

– підтримка локалізації (наприклад, українська, англійська, німецька тощо);

– простий синтаксис та інтеграція з Node.js, Nest.js, Express тощо.

Faker.js ідеально підходить для наповнення записів у базах даних при розробці, генерації тестових даних у тестах, імітації вхідних даних при створенні серверів.

4 ОПИС ПРОГРАМНОЇ РЕАЛІЗАЦІЇ ДОДАТКУ

Програму реалізовано у вигляді клієнтського веб-додатку, який спілкується зі смарт-контрактом в блокчейні та сервером для отримання статистики.

Можливості застосунку подані в якості діаграмі прецедентів на рисунку 4.1.

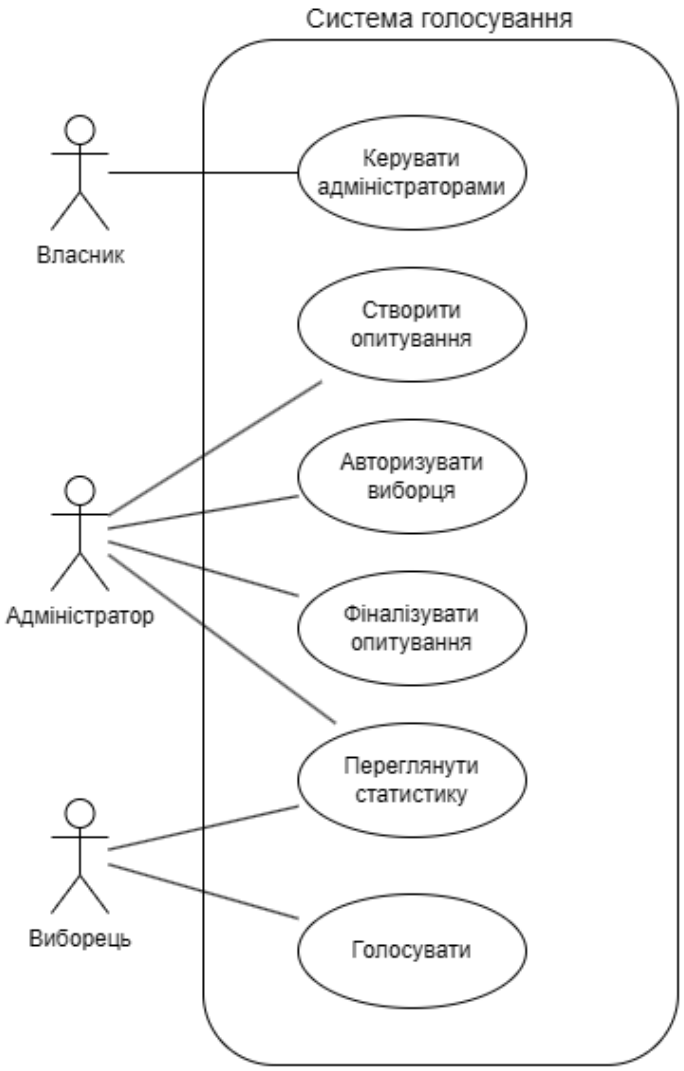

Рисунок 4.1 — Діаграма прецедентів застосунку

Для клієнтської частини використано бібліотеки React.js, Material UI, Ethers.js та браузерне розширення MetaMask. React.js є сучасним рішенням для написання веб-додатків в реактивному стилі, що прискорює розробку. Бібліотека з готовими

компонентами Material UI також додатково пришвидшує розвиток додатку та надає розробникам елементи інтерфейсу, відповідні до сучасних стандартів дизайну. Ethers.js та MetaMask забезпечують з’єднання зі смарт-контрактом та моніторинг блокчейну.

Серверна частина розроблена на основі фреймворку Nest.js та бібліотеки Ethers.js. Модульність Nest.js дозволяє ізолювати окремі компоненти програми та зручно їх інтегрувати між собою. Для тестування також використано бібліотеку Faker.js, що дозволяє швидко імітувати дані, що можуть поступати в систему.

Смарт-контракт розроблено на основі мови Solidity. Для тестування та розгортання використано бібліотеку Hardhat та набір інструментів Foundry.

4.1 Структура смарт-контракту додатку

Смарт-контракт TrustPoll, розроблений на Solidity, є основою системи, відповідаючи за створення опитувань, обробку голосів, авторизацію та підрахунок результатів. Контракт розгорнуто в локальній мережі Foundry Anvil [18] і повністю покритий юніт-тестами через Hardhat [19] і Ethers.js. Код смарт-контракту надано в додатку А.

4.1.1 Основні компоненти контракту

Контракт включає ключові структури, мапінги [20] та події для управління голосуваннями.

Змінні та структури:

– address public owner — адреса власника контракту, який має право додавати/видаляти адміністраторів;

– struct Poll — структура для зберігання даних опитування;

– string name — назва опитування;

– uint256 startTime, uint256 endTime — час початку та завершення;

– bool isFinalized — статус фіналізації;

– mapping(address => bool) voters — список авторизованих виборців;

– mapping(string => uint256) votes — кількість голосів за кожну опцію;

– string[] options — список опцій;

– string winningOption — переможна опція після фіналізації;

– mapping(address => bool) public admins — список адміністраторів;

– mapping(uint256 => Poll) public polls — мапінг опитувань за ідентифікатором;

– uint256 public pollCount — лічильник опитувань.

Події, що виникають під час обробки

– AdminAdded(address indexed admin) — додавання адміністратора;

– AdminDeleted(address indexed admin) — видалення адміністратора;

– PollCreated(uint256 indexed pollId, string name, uint256 startTime, uint256 endTime) — створення опитування;

– VoterAuthorized(uint256 indexed pollId, address indexed voter) — авторизація виборця;

– VoteCast(uint256 indexed pollId, address indexed voter, string option) — фіксація голосу;

– PollFinalized(uint256 indexed pollId, string winningOption) — фіналізація опитування.

Модифікатори функцій:

– onlyOwner() — обмежує доступ до функцій (addAdmin, removeAdmin) власником;

– onlyAdmin() — обмежує доступ до функцій (createPoll, authorizeVoter, finalizePoll) адміністраторами;

– pollExists(uint256 pollId) — перевіряє існування опитування.

Контракт ініціалізується через конструктор, який встановлює власника. Структура Poll дозволяє зберігати дані опитувань, а мапінги забезпечують ефективний доступ до голосів і виборців. Події дозволяють клієнтській та серверній частині реагувати на зміни в реальному часі.

4.1.2 Функціонал контракту

Контракт реалізує ключові функції для управління голосуваннями.

– addAdmin(address admin) — додає нового адміністратора. Доступна лише власнику. Викликає подію AdminAdded;

– removeAdmin(address admin) — видаляє адміністратора. Доступна лише власнику. Викликає подію AdminDeleted;

– createPoll(string memory name, uint256 startTime, uint256 endTime, string[] memory options) — створює нове опитування з унікальним ідентифікатором (pollCount). Доступна адміністраторам. Перевіряє коректність термінів і опцій. Викликає подію PollCreated;

– authorizeVoter(uint256 pollId, address voter) — авторизує виборця для участі в опитуванні. Доступна адміністраторам. Перевіряє існування опитування. Викликає подію VoterAuthorized;

– vote(uint256 pollId, string memory option) — фіксує голос виборця за опцію. Перевіряє авторизацію виборця, існування опції та терміни опитування. Викликає подію VoteCast;

– finalizePoll(uint256 pollId) — фіналізує опитування, визначаючи переможну опцію. Доступна адміністраторам після завершення терміну. Викликає подію PollFinalized.

Функції захищені модифікаторами для запобігання несанкціонованому доступу. Тести перевіряють коректність логіки, включаючи крайові випадки (наприклад, голосування після завершення опитування). Контракт оптимізовано для зменшення витрат на газ шляхом мінімізації операцій із мапінгами.

4.1.3 Тестування смарт-контракту

Тестування смарт-контрактів є критично важливим етапом для забезпечення його надійності, коректності та стійкості до помилок. Цей етап вважається невід’ємним для будь-якого смарт-контракту під час розробки.

Для тестування використано середовище Hardhat у поєднанні з бібліотекою Ethers.js, що дозволило досягти 100% покриття юніт-тестами. Тести проводилися в

локальній тестовій мережі, створеній за допомогою Foundry Anvil, що забезпечило швидке виконання транзакцій. Нижче описано процес тестування, використані інструменти та сценарії перевірки функціоналу контракту.

Hardhat — середовище розробки для Ethereum, що підтримує компіляцію, тестування, розгортання та дебагінг смарт-контрактів. Використано для написання, запуску та аналізу юніт-тестів, а також для створення локального середовища для розгортання контракту.

Ethers.js — бібліотека JavaScript для взаємодії з блокчейном Ethereum, включаючи виклик функцій контракту та обробку подій. Використано для створення екземплярів контракту, виклику його функцій у тестах і перевірки подій.

Foundry Anvil — інструмент для створення локальної тестової мережі Ethereum, сумісної з EVM. Забезпечує середовище для розгортання контракту TrustPoll без витрат на газ, що дозволяє тестувати в ізольованих умовах.

Тестування смарт-контракту TrustPoll проводилося в кілька етапів, щоб забезпечити повне покриття всіх функцій і сценаріїв, включаючи нормальні випадки, крайові ситуації та потенційні помилки.

1. Налаштування тестового середовища Hardhat для використання локальної мережі через Foundry Anvil. Встановлено плагін для інтеграції з Ethers.js для аналізу покриття тестами. Створено тестові акаунти (власник, адміністратори, виборці) з приватними ключами, наданими Anvil.

2. Тести написано в папці test. Кожен тестовий набір відповідає окремій групі функцій контракту (наприклад, "Управління адміністраторами", "Створення опитувань", "Голосування").

3. Перед кожним тестом контракт TrustPoll розгортався в локальній мережі за допомогою Hardhat. Використано Ethers.js для створення екземпляра контракту з різними акаунтами (власник, адміністратор, виборець) для тестування ролей.

4. Тести запускаються командою npx hardhat test, що виконує всі сценарії та генерує звіт про результати. Приклад на рисунку 4.2.

```
> npx hardhat test

Compiled 1 Solidity file successfully (evm target: paris).


  TrustPoll Contract E2E Tests
    √ should execute a complete poll lifecycle successfully (170ms)

  TrustPoll Contract Unit Tests
    Admin Management
      √ should allow owner to add admin
      √ should allow owner to remove admin
      √ should revert if non-owner tries to remove admin
      √ should revert if owner tries to remove self from admins
      √ should revert if non-owner tries to add admin
      √ should revert if adding zero address as admin
      √ should revert if address is already an admin
    Poll Creation
```

Рисунок 4.2 — Виконання тестів смарт-контракту

Таке тестування дозволяє швидко знаходити проблемні місця й налагоджувати код. Окрім цього, цей підхід дозволяє не повторювати помилки під час подальшої розробки та модифікації програм.

Також для більшої надійності використано перевірку на кількість покритих рядків коду смарт-контракту за допомогою команди npx hardhat coverage. Такий метод тестування забезпечує наочну демонстрацію степені покриття коду тестами. Приклад перевірки наведено в рисунку 4.3.

```
  35 passing (2s)

----------------|----------|----------|----------|----------|----------------|
File            |  % Stmts | % Branch |  % Funcs |  % Lines |Uncovered Lines |
----------------|----------|----------|----------|----------|----------------|
 contracts\     |      100 |      100 |      100 |      100 |                |
  TrustPoll.sol |      100 |      100 |      100 |      100 |                |
----------------|----------|----------|----------|----------|----------------|
All files       |      100 |      100 |      100 |      100 |                |
----------------|----------|----------|----------|----------|----------------|

> Istanbul reports written to ./coverage/ and ./coverage.json

Process finished with exit code 0
```

Рисунок 4.3 — Перевірка степені покриття коду смарт-контрактами тестами

Групи перевірок, що були виконані під час тестування:

– ініціалізація контракту;

– управління адміністраторами;

– створення опитувань;

– авторизація виборців;

– голосування;

– фіналізація опитувань;

– отримання даних;

– крайові випадки.

4.2 Серверна частина

Серверна частина, розроблена на Nest.js із TypeScript, виконує роль посередника між клієнтською частиною і смарт-контрактом, забезпечуючи обробку запитів, генерацію тестових даних і статистичний аналіз. Він складається з трьох модулів: blockchain, helper і statistics.

4.2.1 Модуль взаємодії з блокчейном

Модуль blockchain відповідає за взаємодію зі смарт-контрактом через Ethers.js.

Функції:

– підключення до локальної мережі Foundry Anvil або тестовий Ethereum через провайдер Ethers.js;

– ініціалізація контракту TrustPoll із використанням ABI та адреси розгортання;

– виклик функцій контракту (createPoll, vote, finalizePoll) від імені авторизованих акаунтів;

– прослуховування подій (PollCreated, VoteCast) для оновлення стану системи.

Модуль використовує Ethers.js для створення об’єкта контракту та управління транзакціями. Акаунти з приватними ключами (для власника, адмінів, виборців) зберігаються в конфігурації для тестування. Модуль надає сервіси для інших модулів, як-от виклик createPoll для модуля helper.

4.2.2 Допоміжний модуль

Модуль helper відповідає за генерацію тестових даних для прискорення розробки та тестування.

Функції:

– створює кілька опитувань у блокчейні. Використовує Faker.js для генерації назв (наприклад, "Вибори ради", "Опитування громади") і опцій (наприклад, "Кандидат А", "Кандидат Б");

– випадковим чином авторизує виборців і фіксує голоси від їх імені. Використовує Faker.js для рандомізації вибору опцій.

Кінцеві точки викликають функції контракту через модуль blockchain. Наприклад, створює 5–10 опитувань із випадковими параметрами, або генерує 50–100 голосів для кожного опитування. Дані зберігаються в блокчейні, що дозволяє клієнтській частині відображати статистику.

Модуль спрощує тестування, імітуючи реальні сценарії голосування. Використання Faker.js забезпечує реалістичність даних.

4.2.3 Модуль статистики

Модуль statistics обробляє дані для відображення статистики на клієнтській частині.

Функції:

– збір даних про голоси з контракту через функцію getPollOptions і мапінг votes;

– обчислення розподілу голосів для секторної діаграми (відсоток голосів за кожну опцію);

– формування часових рядів для лінійного графіку (кількість голосів за опцію залежно від часу);

– генерація списку останніх голосів для таблиці (адреса виборця, опція, час).

Модуль використовує Ethers.js для отримання подій VoteCast і даних із контракту. Дані агрегуются у форматі JSON і передаються клієнтській частині через REST API (наприклад, GET /statistics/poll/:pollId). Обробка виконується асинхронно для забезпечення реального часу.

Оптимізація запитів до блокчейну зменшує навантаження, а кешування даних прискорює відображення статистики.

4.3 Клієнтська частина

Клієнтська частина, розроблена на React.js із Material UI, забезпечує інтуїтивний інтерфейс для користувачів. Вона складається з трьох вкладок: дешборд (головна), голосування та адмін-панель, і інтегрується з MetaMask для автентифікації.

4.3.1 Вкладка статистики

Дешборд (головна) відображає статистику обраного опитування в реальному часі. Складові дешборду:

– лінійний графік, показує кількість голосів за кожну опцію залежно від часу. Реалізовано за допомогою бібліотеки Material UI, яка отримує дані з модуля statistics;

– секторна діаграма, відображає відсотковий розподіл голосів між опціями. Використовує Material UI для рендерингу;

– таблиця голосів, показує останні голоси (адреса виборця, опція, час). Реалізовано через компонент Material UI.

Дешборд отримує дані через API серверу. React-компоненти оновлюються при зміні стану за допомогою хуків useState і useEffect. Інтеграція з Ethers.js дозволяє прослуховувати подію VoteCast для оновлення графіку в реальному часі. Адаптивний дизайн Material UI забезпечує зручність на різних пристроях. Дані кешуються для зменшення запитів до бекенду.

4.3.2 Вкладка голосування

Вкладка дозволяє користувачам переглядати опитування, голосувати та створювати нові опитування (для адміністраторів).

Складові:

– таблиця опитувань відображає список опитувань (назва, терміни, статус);

– кнопки голосування дозволяють виборцям обирати опцію та підписувати транзакцію через MetaMask, викликає функцію vote контракту;

– секція створення опитувань доступна адміністраторам, форма для введення назви, термінів і опцій, викликає createPoll через Ethers.js;

– секція авторизації виборців, доступна адміністраторам, форма для введення адрес виборців. Викликає authorizeVoter;

– результати після фіналізації відображають переможну опцію (через getPollResult).

Компоненти використовують Ethers.js для взаємодії з контрактом і бекендом для отримання списку опитувань. Події PollCreated і PollFinalized оновлюють таблицю в реальному часі.

Перевірка ролей (виборець/адміністратор) через контракт (мапінг admins) обмежує доступ до функцій для звичайних користувачів.

4.3.3 Адмін-панель

Адмін-панель доступна лише власнику контракту та адміністраторам для управління адміністраторами.

Панель складається з двох компонентів:

– таблиця адмінів показує список адміністраторів (адреси, статус);

– форма додавання/видалення адмінів доступна лише власнику, викликає функції addAdmin і removeAdmin через Ethers.js.

Приклад реалізації адмін-панелі можна побачити на рисунку 4.4.

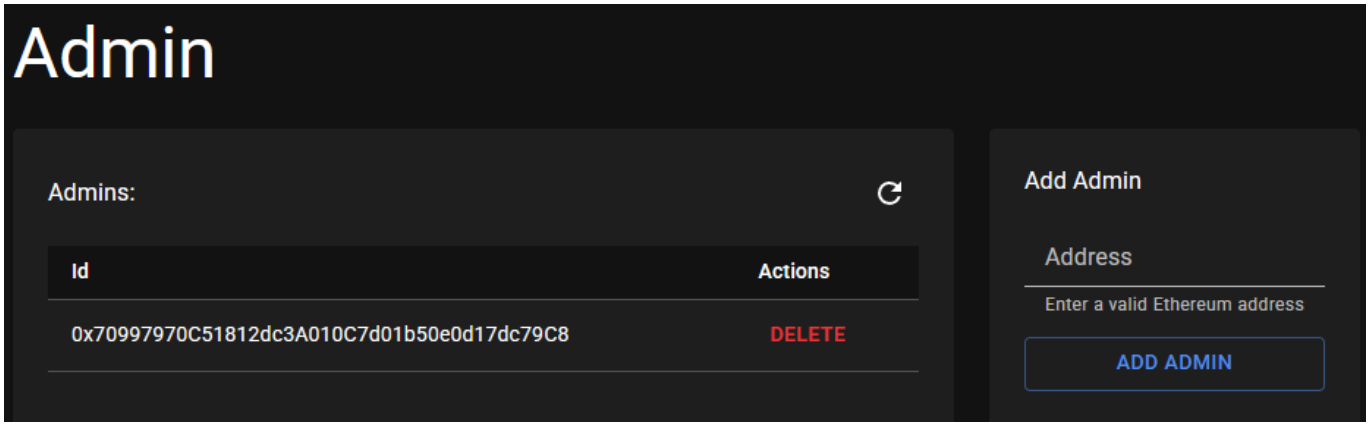

Рисунок 4.4 — Інтерфейс адмін-панелі

Панель перевіряє статус користувача через контракт. Події AdminAdded і AdminDeleted оновлюють таблицю. Форми використовують Material UI для стилізації та валідації адрес. Захист доступу через модифікатор onlyOwner забезпечує безпеку управління.

5 ВЗАЄМОДІЯ КОРИСТУВАЧА З ДОДАТКОМ

Додаток забезпечує інтуїтивний інтерфейс для трьох типів користувачів: власника контракту, адміністраторів і виборців. Взаємодія здійснюється через клієнтську частину на React.js із Material UI, інтегрований із MetaMask для автентифікації та смарт-контрактом TrustPoll для обробки голосувань.

5.1 Системні вимоги та налаштування

Системні вимоги для клієнтів і адміністраторів є мінімальними, оскільки додаток є веб-застосунком, доступним через браузер. Однак для стабільної роботи потрібне відповідне апаратне забезпечення та стабільне підключення до Інтернету:

– процесор — 2 ГГц або швидший (наприклад, Intel Core i3, AMD Ryzen 3 або еквівалент);

– оперативна пам’ять — мінімум 4 ГБ (рекомендується 8 ГБ для комфортної роботи з браузером і MetaMask);

– вільне місце на диску — мінімум 500 МБ для встановлення браузера і MetaMask;

– розширення екрану — 1280x720 або вище для зручного відображення інтерфейсу та графіків;

– пристрої — настільний ПК, ноутбук або планшет із підтримкою сучасних браузерів;

– операційна система Windows 10, macOS, Linux, Android або інша з підтримкою сучасних браузерів та розширення MetaMask.

Для роботи з додатком користувачі повинні встановити мінімальний набір програмного забезпечення, яке забезпечує доступ до веб-інтерфейсу та взаємодію з блокчейном: веб-браузер, додаток для браузерів MetaMask.

5.1.1 Веб-браузер

Необхідний сучасний веб-браузер, який підтримує JavaScript і Web3 API для взаємодії з MetaMask. Рекомендовані браузери:

– Google Chrome (версія 90 або новіша);

– Mozilla Firefox (версія 89 або новіша);

– Microsoft Edge (версія 91 або новіша);

– Brave (версія 1.40 або новіша).

Браузер відображає фронтенд додатку, побудований на React.js із Material UI, і забезпечує доступ до смарт-контракту через MetaMask.

5.1.2 Додаток MetaMask

Для авторизації користувач має встановити криптогаманець і розширення MetaMask для браузера, яке дозволяє користувачам автентифікуватися, підписувати транзакції та взаємодіяти з блокчейном Ethereum. Завантажити розширення MetaMask із офіційного сайту (metamask.io) або через магазин розширень браузера (Chrome Web Store, Firefox Add-ons).

Після встановлення необхідно створити або імпортувати гаманець із приватним ключем чи мнемонічною фразою.

5.2 Перегляд статистики

Головною сторінкою веб-додатку для децентралізованого голосування є дешборд, який дозволяє користувачам (виборцям і адміністраторам) переглядати статистику опитувань у реальному часі. Дешборд відображає дані у вигляді лінійного графіка, що забезпечує наочне представлення результатів голосування. Приклад зображено на рисунку 5.1.

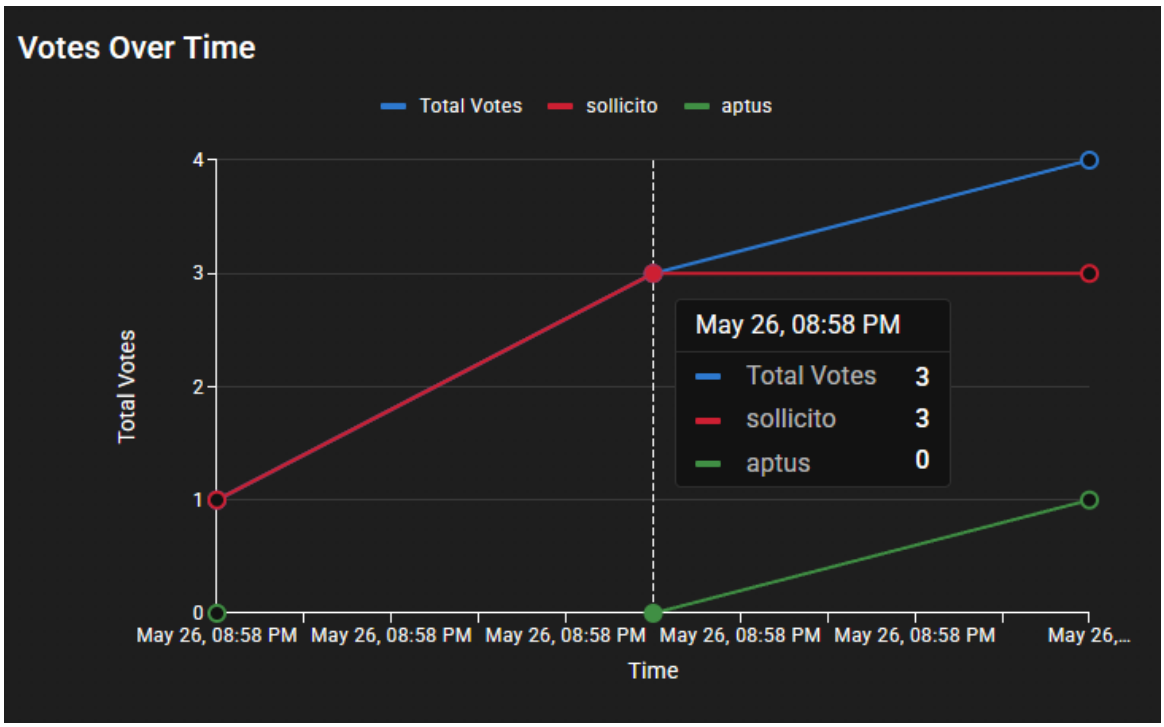

Рисунок 5.1 — Лінійний графік голосів на дешборді

Також для більшої зручності, користувач може переглянути секторну діаграму за обраним опитуванням з голосами відповідно за кожну опцію. Приклад наведено в рисунку 5.2.

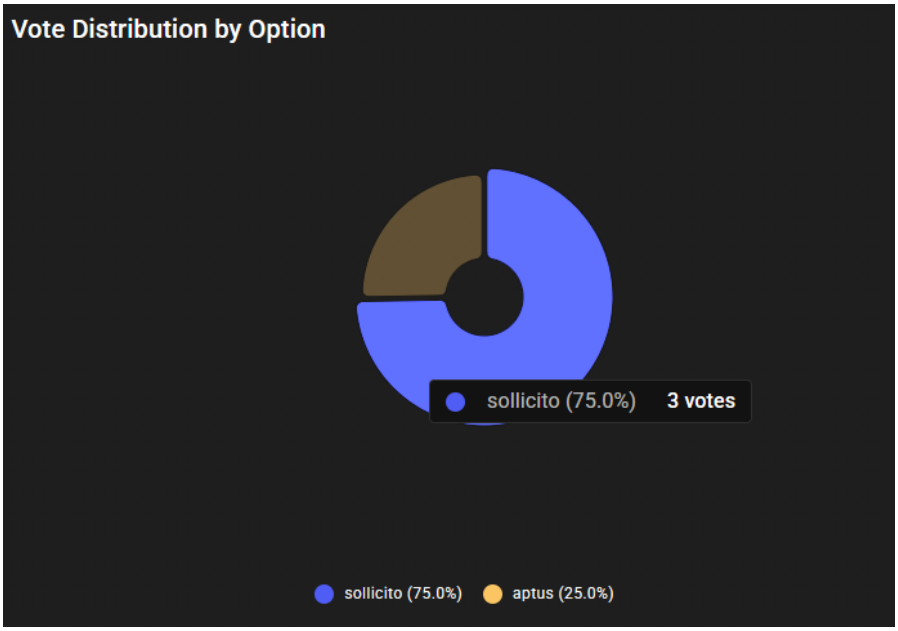

Рисунок 5.2 — Секторна діаграма голосів в опитуванні

Користувач може перемикатися між опитуваннями, обираючи інше опитування зі списку. При кожному виборі дешборд оновлює всі компоненти. Якщо опитування не містить голосів або ще не розпочалося, дешборд відображає повідомлення. Дешборд є інтуїтивним завдяки чистому дизайну Material UI, який забезпечує єдиний стиль компонентів і адаптивність для різних пристроїв.

5.3 Автентифікація користувачів

Усі користувачі (власник, адміністратори, виборці) взаємодіють із додатком через веб-інтерфейс, підключаючись до блокчейну Ethereum за допомогою MetaMask. Автентифікація є першим кроком для доступу до функціоналу.

Процес авторизації за допомогою MetaMask:

– користувач відкриває додаток у браузері з установленним розширенням MetaMask;

– на головній сторінці натискає кнопку "Підключити гаманець", що викликає діалог MetaMask;

– розширення MetaMask запитує дозвіл на підключення до сайту, після чого користувач підтверджує свою Ethereum-адресу;

– клієнтська частина перевіряє роль користувача через смарт-контракт TrustPoll;

– якщо адреса є owner, надається доступ до адмін-панелі;

– якщо адреса є в мапінгу admins, надається доступ до функцій адміністратора;

– інакше користувач вважається виборцем із доступом до голосування та статистики;

– після успішної автентифікації користувач перенаправляється на дешборд.

Приклад авторизації наведену на рисунку 5.3.

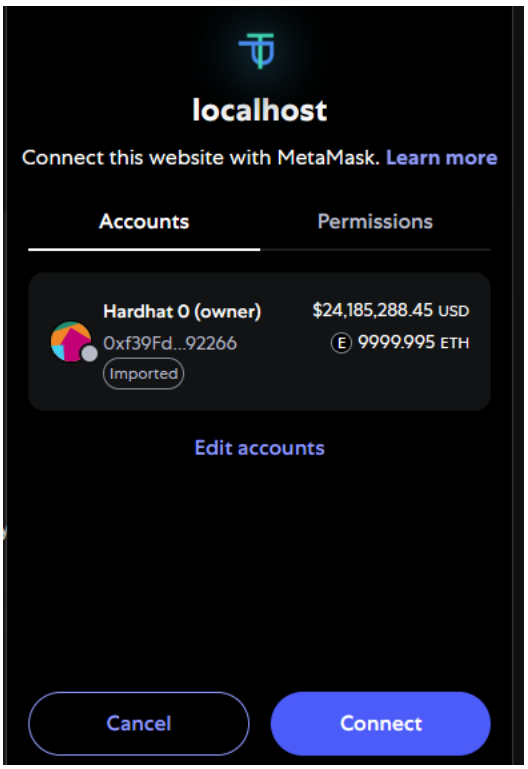

Рисунок 5.3 — Інтерфейс додатку MetaMask при під’єднанні

Клієнтська частина використовує бібліотеку Ethers.js для взаємодії з MetaMask через Web3Provider. Він відстежує стан підключення та оновлює інтерфейс відповідно до ролі користувача. У разі помилки (наприклад, відсутність MetaMask) відображається повідомлення з інструкцією щодо встановлення.

5.4 Голосування користувачем

Виборець використовує вкладку "Голосування" (Voting) для перегляду доступних опитувань, вибору опції та фіксації голосу в блокчейні.

Послідовність дій:

– після автентифікації через MetaMask виборець переходить на вкладку "Голосування";

– відображається таблиця з опитуваннями, що включає назву, терміни та статус (активне/завершене);

– виборець обирає активальне опитування, натискаючи на нього, і бачить список опцій (отриманих через getPollOptions);

– користувач натискає кнопку "Голосувати" біля обраної опції, що викликає діалог MetaMask для підписування транзакції;

– MetaMask надсилає транзакцію до смарт-контракту, викликаючи функцію vote(pollId, option). Контракт перевіряє авторизацію виборця та фіксує голос;

– після підтвердження транзакції фронтенд оновлює таблицю, а подія VoteCast сигналізує бекенду для оновлення статистики;

– виборець може перейти на дешборд, щоб переглянути оновлені графіки та таблиці.

Вікно голосування зображено на рисунку 5.4.

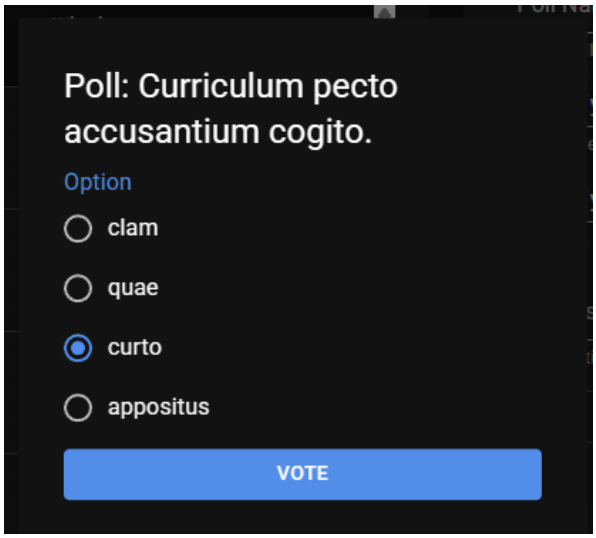

Рисунок 5.4 — Вікно голосування

Перевірка статусу опитування виконується через контракт, запобігаючи голосуванню після закінчення терміну. Інтерфейс інформує користувача про успішне голосування або помилки.

Асинхронна обробка транзакцій через Ethers.js забезпечує швидке оновлення користувацького інтерфейсу.

5.5 Адміністрування

Адміністратори та власник використовують вкладки "Голосування" та "Адмін-панель" для створення опитувань, авторизації виборців, фіналізації результатів і управління адміністраторами.

Процес створення опитування:

– адміністратор автентифікується через MetaMask і переходить на вкладку "Голосування";

– у секції "Створити опитування" заповнює форму: назва, дати початку та завершення, список опцій (наприклад, "Кандидат А", "Кандидат Б");

– після натискання "Створити" фронтенд викликає функцію createPoll через Ethers.js, а MetaMask підписує транзакцію;

– контракт додає опитування до мапінгу polls і випускає подію PollCreated;

– таблиця опитувань оновлюється в реальному часі.

Процес авторизації виборців:

– адміністратор у секції "Авторизація виборців" вводить Ethereum-адреси виборців;

– натискає "Авторизувати", викликаючи authorizeVoter через MetaMask;

– контракт оновлює мапінг voters і випускає подію VoterAuthorized;

– список авторизованих виборців відображається в інтерфейсі.

ВИСНОВКИ

На основі аналізу підходів до створення систем голосування обґрунтовано використання блокчейн-технологій із застосуванням смарт-контрактів для децентралізованої системи голосування. Децентралізована архітектура усуває потребу в посередниках, забезпечуючи незмінність даних і публічну верифікацію транзакцій. Порівняння з централізованими системами показало, що блокчейн-підхід підвищує довіру завдяки криптографічному захисту й аудіюванню.

За аналізом програмних засобів обґрунтовано вибір технологій, що включає мову Solidity для смарт-контракту, фреймворк Nest.js для серверної частини, бібліотеку React.js із Material UI для клієнтського інтерфейсу та Ethers.js для взаємодії з блокчейном. Мова Solidity забезпечила реалізацію складної логіки голосування, тоді як Nest.js дозволив створити модульну серверну частину з ефективною обробкою даних і аналітики. React.js із Material UI забезпечив інтуїтивний і адаптивний інтерфейс, а Ethers.js спростив інтеграцію з блокчейном. Використання Hardhat і Foundry Anvil для тестування та локального розгортання, а також Faker.js для генерації тестових даних, прискорило розробку та підвищило надійність системи.

Розроблено додаток для децентралізованого голосування. Система додатку охоплює смарт-контракт, серверну частину з модулями для взаємодії з блокчейном, обробки статистики та генерації тестових даних, а також клієнтський веб-інтерфейс. Система забезпечує повний цикл голосування: від створення опитувань адміністраторами до перегляду результатів виборцями через графіки й таблиці. Інтеграція з криптогаманцем спрощує автентифікацію та підписання транзакцій, роблячи додаток доступним для користувачів без технічних навичок.

На основі тестування доведено коректність роботи розробленого програмного забезпечення. Тести смарт-контракту, проведені за допомогою Hardhat, досягли 100% покриття коду, підтвердивши надійність функцій створення опитувань, голосування та фіналізації. Результати тестування підтвердили, що система працює без збоїв, забезпечує точність підрахунку голосів і стійкість до помилок.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Селдин Дж. На нову хвилю дезінформації очікують перед виборами в США. *Голос Америки Українською*. URL: https://www.holosameryky.com/a/new-wave-of-disinformation-expected-on-eve-of-us-election/6823542.html (дата звернення: 18.05.2025).

2. About Voting Systems. *Florida Department of State*. URL: https://dos.fl.gov/elections/voting-systems/about-voting-systems/ (дата звернення: 26.05.2025).

3. Antoniou A. A Trust-Centered Approach for Building E-Voting Systems. *ResearchGate*. URL: https://www.researchgate.net/publication/221561352_A_Trust-Centered_Approach_for_Building_E-Voting_Systems (дата звернення: 26.05.2025).

4. Y D., Aradhya K. S., Ms H. A decentralized architecture for transparent e-voting using blockchain and multi-factor authentication. *IRJMETS*. URL: https://www.irjmets.com/uploadedfiles/paper//issue_4_april_2025/72960/final/fin_irjmets1744919019.pdf (дата звернення: 26.05.2025).

5. Taş R. A Systematic Review of Challenges and Opportunities of Blockchain for E-Voting. *MDPI*. URL: https://www.mdpi.com/2073-8994/12/8/1328 (дата звернення: 26.05.2025).

6. Muskan, Surbhi, Kaur K. A Study On Decentralized Voting System. *IRJET-International Research Journal of Engineering and Technology*. URL: https://www.irjet.net/archives/V10/i7/IRJET-V10I7170.pdf (дата звернення: 26.05.2025).

7. Blockchain-enhanced electoral integrity: a robust model for secure digital voting systems in Oman / A. Shaikh та ін. *F1000Research*. URL: https://f1000research.com/articles/14-223 (дата звернення: 18.05.2025).

8. GeeksforGeeks. Blockchain Structure - GeeksforGeeks. *GeeksforGeeks*. URL: https://www.geeksforgeeks.org/blockchain-structure/ (дата звернення: 18.05.2025).

9. BasuMallick C. Smart contracts: types, benefits, and tools | spiceworks - spiceworks. *Spiceworks Inc*. URL: https://www.spiceworks.com/tech/innovation/articles/what-are-smart-contracts/ (дата звернення: 18.05.2025).

10. Documentation - TypeScript for the New Programmer. *TypeScript: JavaScript With Syntax For Types*. URL: https://www.typescriptlang.org/docs/handbook/typescript-from-scratch.html (дата звернення: 18.05.2025).

11. React reference overview – react. *React*. URL: https://react.dev/reference/react (дата звернення: 19.05.2025).

12. Overview - material UI. *MUI: The React component library you always wanted*. URL: https://mui.com/material-ui/getting-started/ (дата звернення: 18.05.2025).

13. Documentation. URL: https://docs.ethers.org/v6/ (дата звернення: 18.05.2025).

14. React.js & Ethers.js: Demo of using the InfuraProvider vs the Metamask wallet Web3Provider | MetaMask Help Center. *MetaMask Help Center*. URL: https://support.metamask.io/develop/building-with-infura/javascript-typescript/infuraprovider-metamaskwalletprovider-react-ethersjs/ (дата звернення: 18.05.2025).

15. Documentation | NestJS - A progressive Node.js framework. *Documentation | NestJS*. URL: https://docs.nestjs.com/ (дата звернення: 18.05.2025).

16. Aiyelabegan T. A guide on dependency injection in nestjs. *DigitalOcean*. URL: https://www.digitalocean.com/community/tutorials/a-guide-on-dependency-injection-in-nestjs (дата звернення: 18.05.2025).

17. Getting started | faker. *Faker*. URL: https://fakerjs.dev/guide/ (дата звернення: 18.05.2025).

18. Foundry Book. *Introduction - Foundry Book*. URL: https://book.getfoundry.sh/anvil/ (дата звернення: 18.05.2025).

19. Getting started with Hardhat | Ethereum development environment for professionals by Nomic Foundation. *Hardhat*. URL: https://hardhat.org/hardhat-runner/docs/getting-started (дата звернення: 18.05.2025).

20. 0xjoi. Understanding Hash Functions in Solidity: A Deep Dive into Data Integrity and Security. Medium. URL: https://medium.com/@joichiro.sai/understanding-hash-functions-in-solidity-a-deep-dive-into-data-integrity-and-security-c617d095b02c (дата звернення: 26.05.2025).

ДОДАТОК А

Код смарт-контракту голосування

Програмні засоби:

– мова програмування — Solidity;

– середовище розробки — Visual Studio Code;

– бібліотека для тестування та розгортки — Hardhat.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract TrustPoll {
    address public owner;

    struct Poll {
        string name;
        uint256 startTime;
        uint256 endTime;
        bool isFinalized;
        mapping(address => bool) voters;
        mapping(string => uint256) votes;
        string[] options;
        string winningOption;
    }

    mapping(address => bool) public admins;
    mapping(uint256 => Poll) public polls;
    uint256 public pollCount;

    event AdminAdded(address indexed admin);
```

```
    event AdminDeleted(address indexed admin);
    event PollCreated(uint256 indexed pollId, string name, uint256 startTime, uint256
endTime);
    event VoterAuthorized(uint256 indexed pollId, address indexed voter);
    event VoteCast(uint256 indexed pollId, address indexed voter, string option);
    event PollFinalized(uint256 indexed pollId, string winningOption);

    modifier onlyOwner() {
        require(msg.sender == owner, "Only owner can call this function");
        _;
    }

    modifier onlyAdmin() {
        require(admins[msg.sender], "Only admin can call this function");
        _;
    }

    modifier pollExists(uint256 pollId) {
        require(pollId < pollCount, "Poll does not exist");
        _;
    }

    constructor() {
        owner = msg.sender;
        admins[msg.sender] = true;
    }

    function addAdmin(address admin) external onlyOwner {
        require(admin != address(0), "Invalid address");
        require(!admins[admin], "Address is already an admin");
```

```
    admins[admin] = true;
    emit AdminAdded(admin);
}

function removeAdmin(address admin) external onlyOwner {
    require(admin != address(0) && admin != owner, "Invalid address");
    delete admins[admin];
    emit AdminDeleted(admin);
}

function createPoll(
    string memory name,
    uint256 startTime,
    uint256 endTime,
    string[] memory options
) external onlyAdmin returns (uint256) {
    require(startTime > block.timestamp, "Start time must be in future");
    require(endTime > startTime, "End time must be after start time");
    require(options.length > 0, "Options cannot be empty");

    Poll storage newPoll = polls[pollCount];
    newPoll.name = name;
    newPoll.startTime = startTime;
    newPoll.endTime = endTime;
    newPoll.isFinalized = false;

    for (uint256 i = 0; i < options.length; i++) {
        newPoll.options.push(options[i]);
        newPoll.votes[options[i]] = 0;
    }
```

```
        emit PollCreated(pollCount, name, startTime, endTime);
        pollCount++;

        return pollCount-1;
    }

    function authorizeVoter(uint256 pollId, address voter) external onlyAdmin
pollExists(pollId) {
        require(voter != address(0), "Invalid voter address");
        Poll storage poll = polls[pollId];
        require(!poll.isFinalized, "Poll is finalized");
        poll.voters[voter] = true;
        emit VoterAuthorized(pollId, voter);
    }

    function vote(uint256 pollId, string memory option) external pollExists(pollId) {
        Poll storage poll = polls[pollId];
        require(!poll.isFinalized, "Poll is finalized");
        require(block.timestamp >= poll.startTime, "Poll has not started");
        require(block.timestamp <= poll.endTime, "Poll has ended");
        require(poll.voters[msg.sender], "Not authorized to vote");

        bool validOption = false;
        for (uint256 i = 0; i < poll.options.length; i++) {
            if (keccak256(abi.encodePacked(poll.options[i])) ==
keccak256(abi.encodePacked(option))) {
                validOption = true;
                break;
            }
```

```
    }
    require(validOption, "Invalid option");

    poll.votes[option]++;
    poll.voters[msg.sender] = false; // Prevent multiple votes
    emit VoteCast(pollId, msg.sender, option);
}

function finalizePoll(uint256 pollId) external onlyAdmin pollExists(pollId) {
    Poll storage poll = polls[pollId];
    require(!poll.isFinalized, "Poll is already finalized");
    require(block.timestamp > poll.endTime, "Poll end time not reached");

    poll.isFinalized = true;

    string memory winningOption;
    uint256 highestVotes = 0;

    for (uint256 i = 0; i < poll.options.length; i++) {
        if (poll.votes[poll.options[i]] > highestVotes) {
            highestVotes = poll.votes[poll.options[i]];
            winningOption = poll.options[i];
        }
    }

    poll.winningOption = winningOption;
    emit PollFinalized(pollId, winningOption);
}
```

```
    function getPollOptions(uint256 pollId) external view pollExists(pollId) returns (string[] memory) {
        return polls[pollId].options;
    }

    function getPollResult(uint256 pollId) external view pollExists(pollId) returns (string memory) {
        return polls[pollId].winningOption;
    }
}
```